UNIT 2 – URLs, Views, Templates & Static Files

Exhaustive Study Notes covering Routing, Dynamic Views, DTL Syntax, Inheritance & Static Assets

1. URL Routing

URL routing is the mechanism of dispatching incoming HTTP requests to specific view handlers based on URL patterns. Django uses clean, human-readable URL schemes without trailing query strings or file extensions.

2. path()

The `path()` function defines URL pattern mappings. Syntax: `path(route, view, kwargs=None, name=None)`. Path Converters automatically capture and parse parameters: • `<str:name>`: Matches any non-empty string (excluding '/'). Default converter. • `<int:id>`: Matches zero or any positive integer and converts to Python `int`. • `<slug:slug>`: Matches alphanumeric string with hyphens/underscores. • `<uuid:id>`: Matches formatted UUID string. • `<path:subpath>`: Matches non-empty string including forward slashes.

```
from django.urls import path
from . import views

urlpatterns = [
    path('student/<int:id>/', views.student_detail, name='student_detail'),
    path('article/<slug:title_slug>/', views.article_view, name='article_view'),
]
```

Django URL Routing & View Resolution Dispatcher

Diagram: *Django URL Routing & View Resolution Dispatcher.*

3. include()

The `include()` function allows referencing other URLconf modules (app-level `urls.py`). It enables modularity and app decoupling.

```
# Project-level urls.py
from django.urls import path, include

urlpatterns = [
    path('students/', include('students.urls', namespace='students')),
]
```

4. Function-Based Views (FBVs)

A Function-Based View is a Python function that receives `HttpRequest` as its first argument and returns an `HttpResponse` object.

```
from django.http import HttpResponse, Http404

def student_profile(request, id):
    if id <= 0:
        raise Http404("Student ID does not exist")
    return HttpResponse(f"Displaying Profile for Student ID: {id}")
```

5. HttpResponse

The `HttpResponse` class represents the outgoing HTTP response. It carries HTTP status codes, headers, and body content.

```
from django.http import HttpResponse, JsonResponse

def custom_response(request):
    response = HttpResponse("Resource Created", status=201)
    response['X-Custom-Header'] = 'DjangoPortal'
    return response
```

6. render()

The `render()` function is a convenience shortcut that combines a specified HTML template with a context dictionary and returns an `HttpResponse` containing the rendered HTML string.

```
from django.shortcuts import render

def home(request):
    context = {'user_role': 'Administrator', 'active_tab': 'Dashboard'}
    return render(request, 'home.html', context)
```

7. View to Template Data

Data is passed from views to templates using a Python dictionary called context. Keys in the context dictionary become variable names inside the template.

8. Django Templates

Django Template Language (DTL) separates application business logic from presentation UI layout. DTL is deliberately non-executable to prevent execution of arbitrary code inside templates.

9. Template Variables

Variables are outputted using double curly braces: `{{ variable_name }}`. Django evaluates variable attributes in order: Dictionary key -> Attribute lookup -> Method call -> List index.

```
<!-- Template variable evaluation example -->
<p>Welcome, {{ student.first_name }}!</p>
<p>Email: {{ student.email }}</p>
<p>Marks: {{ student.get_total_marks }}</p>
```

10. Template Tags

Template tags perform execution logic, loops, and conditional branching. Syntax: `{% tag_name %}`.

11. if Condition

The `{% if %}` tag evaluates Boolean expressions and renders block content accordingly.

```
{% if marks >= 75 %}
    <span class="badge success">Distinction</span>
{% elif marks >= 40 %}
    <span class="badge info">Pass</span>
{% else %}
    <span class="badge danger">Fail</span>
{% endif %}
```

12. for Loop

The `{% for %}` tag iterates over any sequence. It provides loop context variables like `forloop.counter`, `forloop.first`, `forloop.last`.

```
<ul>
{% for student in student_list %}
    <li>{{ forloop.counter }}. {{ student.name }} ({{ student.email }})</li>
{% empty %}
    <li>No active students registered.</li>
{% endfor %}
</ul>
```

13. Template Filters

Filters alter variable output formatting. Pipe symbol `|` is used to apply filters.

```
{{ student.name|upper }}
{{ student.joined_date|date:"F d, Y" }}
{{ student.bio|default:"No bio available"|truncatewords:15 }}
```

14. Template Inheritance

Template inheritance allows defining a master layout template (`base.html`) containing common UI elements (headers, navigation, footers) and placeholder blocks (`{% block %}`). Child templates inherit the master layout and override specific blocks.

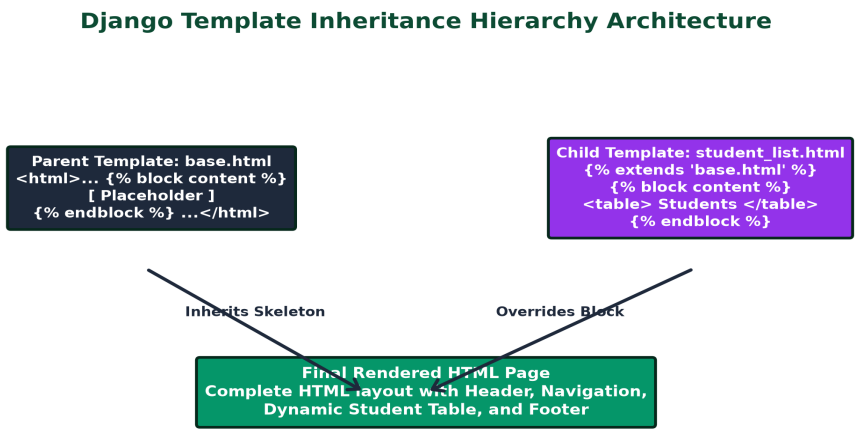

Diagram: *Django Template Inheritance Hierarchy Architecture.*

15. base.html

Sample standard master layout pattern `base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>{% block title %}Smart Portal{% endblock %}</title>
    {% load static %}
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
    <header><h1>Dashboard Header</h1></header>
    <main>
        {% block content %}{% endblock %}
    </main>
    <footer>&copy; 2026 Django Smart Notes</footer>
</body>
</html>
```

16. Static Files

Static files include CSS stylesheets, JavaScript files, icons, and logos. Django handles static files using `STATIC_URL`, `STATICFILES_DIRS`, and `{% load static %}`.

```
# settings.py
STATIC_URL = '/static/'
STATICFILES_DIRS = [os.path.join(BASE_DIR, 'static')]

# Template Usage
{% load static %}
<img src="{% static 'images/logo.png' %}" alt="Logo">
```

17. GET Request

HTTP GET requests retrieve data. Parameters sent in the URL query string are accessible via `request.GET` (a `QueryDict` object).

```
def search_view(request):
    query = request.GET.get('q', '') # Read query param 'q'
    return HttpResponse(f"Search results for: {query}")
```

18. POST Request

HTTP POST requests submit data that modifies server-side state (e.g. form submissions). Form data is extracted via `request.POST`.

19. CSRF Protection

Cross-Site Request Forgery (CSRF) protection prevents malicious sites from executing unauthorized commands on behalf of an authenticated user. Django embeds a secret token in user cookies and requires forms to submit `{% csrf_token %}`.

```
<form method="POST" action="/submit/">
    {% csrf_token %}
    <input type="text" name="student_name">
    <button type="submit">Submit</button>
</form>
```

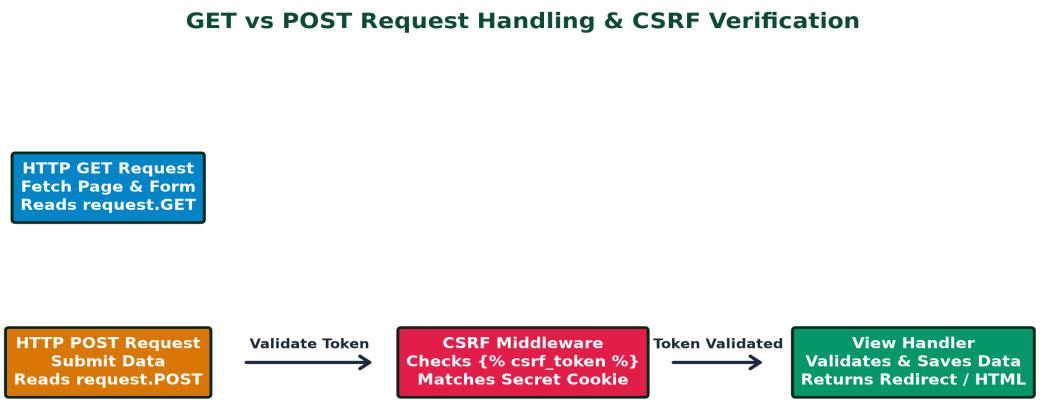

Diagram: *GET vs POST Request Flow and CSRF Security Middleware Token Verification.*

20. Class-Based Views (CBVs)

Class-Based Views organize request handlers into object-oriented classes. Built-in generic CBVs like `ListView`, `DetailView`, `CreateView`, `UpdateView`, `DeleteView` eliminate CRUD boilerplate.

```
from django.views.generic import ListView
from .models import Student

class StudentListView(ListView):
    model = Student
    template_name = 'student_list.html'
    context_object_name = 'students'
```